

Comparative Analysis of Multinomial Logistic Regression and Linear Support Vector Machine for Wine Cultivar Classification

Luke Joaquin Bernardo
College of Computer Studies and Engineering
Jose Rizal University
Taguig City, Philippines
lukejoaquin.bernardo@my.jru.edu

Abstract—This study compares two classical linear models for classification — Multinomial Logistic Regression (MLR) and Linear Support Vector Machine (Linear SVM) — on the UCI Wine dataset for multiclass prediction of wine cultivars from 13 chemical properties. The models were tuned via GridSearchCV with 10-fold stratified cross-validation over a broad spectrum of regularization strengths ($C \in [10^{-4}, 10^4]$) and other hyperparameters. Experimental results demonstrate that MLR achieved perfect classification on the held-out test set (Accuracy = 1.000, κ = 1.000, F1 = 1.000) at its optimal $C = 0.01$, while Linear SVM reached 97.22% accuracy ($\kappa = 0.958$, F1 = 0.972) at $C \approx 3.162$ with hinge loss. This suggests that the probabilistic softmax distribution of MLR learns a better generalization than the geometric margin maximization of Linear SVM for this chemometric problem.

Keywords—Multinomial Logistic Regression, Support Vector Machine, Wine Classification, Hyperparameter Tuning, GridSearchCV, Regularization, Softmax, Margin Maximization.

I. INTRODUCTION

Multiclass classification is a fundamental task of supervised machine learning, where each classifier has to classify the observations into one of three or more classes that are mutually exclusive. Multiclass classification problems are usually generalized from binary classification or ensemble-based approaches like one-vs-rest decomposition [4].

Another pillar of multiclass classification is *chemical profiling*, where substances or specimens are automatically identified via quantitative tests of their physicochemical properties. A common dataset in this vein is the UCI Wine dataset [2]: 178 samples of Italian wines from three varieties (classes 0, 1, and 2) describe thirteen continuous chemical attributes, such as alcohol, malic acid, ash, alkalinity, magnesium, phenols, flavonoids, etc. Previous studies have suggested that machine learning classifiers can be effective for such datasets [6].

We aim to perform two separate experiments using these chemical samples, first to implement and fine-tune two different linear classifiers — a *probabilistic* model (Multinomial Logistic Regression, MLR) and a *geometric* model (Linear SVM). Second, to compare their generalization performance and determine the effect of the regularization parameter C on both models.

The probabilistic and geometric approaches to classification are separate [4]. As described in [3], MLR uses the *softmax* function to estimate class membership probabilities and minimizes cross-entropy loss regularized by C^{-1} . Linear SVM seeks the *maximum-margin hyperplane* that best separates classes geometrically [3], with C defining the trade-off between margin width and misclassification tolerance [5]. Understanding when one of these approaches shines is relevant to the designers of real-world classification systems.

II. METHODOLOGY

A. Software Environment

All experiments were implemented in Python using the following library versions [1]:

- scikit-learn 1.6.1
- numpy 2.0.2
- pandas 2.2.2
- Python 3.12.0

B. Dataset and Partitioning

The `sklearn.datasets.load_wine()` dataset was used [2], comprising 178 samples across 3 wine cultivar classes and 13 chemical feature dimensions. The class distribution is: Class 0 (59 samples), Class 1 (71 samples), and Class 2 (48 samples).

The dataset was partitioned using an 80/20 stratified split with `random_state=42`, producing:

- **Training set:** 142 samples (used for cross-validation and hyperparameter search).
- **Test set:** 36 samples (held out entirely for final evaluation).

Stratification ensures that each split preserves the original class proportions, preventing skewed evaluation results [4].

C. Preprocessing and Pipeline

All features were standardized using `StandardScaler` (zero mean, unit variance), which is a requirement for distance-sensitive models such as SVM [1]. To prevent *data leakage* — where test-set information inadvertently influences the scaler fit — standardization was integrated into a `sklearn.pipeline.Pipeline` for each model [1]. The

scaler is therefore fitted only on training folds during cross-validation and applied to validation/test folds without refitting.

III. EXPERIMENTAL DESIGN

A. Model A: Multinomial Logistic Regression (MLR)

1) *Algorithm Description*: Multinomial Logistic Regression generalizes binary logistic regression to $K > 2$ classes [4]. For an input vector $\mathbf{x} \in \mathbb{R}^d$, the model computes a score $z_k = \mathbf{w}_k^\top \mathbf{x} + b_k$ for each class k , and then applies the **softmax** function to convert scores into probabilities [3]:

$$P(y = k | \mathbf{x}) = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}} \quad (1)$$

The predicted class is the one with the highest probability. Training minimizes the cross-entropy loss with L2 regularization, controlled by the parameter C (inverse of regularization strength):

$$\mathcal{L} = - \sum_{i=1}^N \log P(y_i | \mathbf{x}_i) + \frac{1}{C} \|\mathbf{w}\|^2 \quad (2)$$

A smaller C imposes stronger regularization, leading to a simpler, more biased model. A larger C relaxes regularization, allowing the model to fit the training data more closely but risking overfitting.

The lbfgs solver was used, which natively handles multinomial loss. In scikit-learn 1.6.1, the `multi_class` parameter has been removed as lbfgs defaults to multinomial behavior for multi-class problems.

2) *Hyperparameter Search Space*: The following hyperparameters were searched using GridSearchCV with 10-fold stratified cross-validation:

- $C \in \{10^{-4}, 10^{-3.5}, \dots, 10^4\}$ (17 values, log-spaced via `np.logspace(-4, 4, 17)`)
- $\text{tol} \in \{10^{-4}, 10^{-3}, 10^{-2}\}$
- $\text{class_weight} \in \{\text{None}, \text{'balanced'}\}$

This yields $17 \times 3 \times 2 = 102$ hyperparameter combinations.

3) *Effect of Regularization Parameter C on MLR*: As shown in Fig. 1, very small values of C (e.g., $C = 10^{-4}$) cause severe underfitting, with CV accuracy near 40% — essentially no better than random guessing on a 3-class problem. This is consistent with [3] that high regularization forces the model toward a highly biased, oversimplified decision boundary [5].

As C increases past 10^{-2} , accuracy rises sharply. The optimal performance for MLR was achieved at $C = 0.01$, with mean CV accuracy of 98.62% ($\sigma = 2.76\%$). Beyond this point, accuracy plateaus and eventually shows marginal decline at very large C (e.g., $C = 10^4$), where the training accuracy reaches 100% but CV accuracy drops slightly — a hallmark of overfitting, consistent with the bias-variance tradeoff discussed in [3] and formalized in [5].

B. Model B: Linear Support Vector Machine (Linear SVM)

1) *Algorithm Description*: The Linear SVM seeks a hyperplane $\mathbf{w}^\top \mathbf{x} + b = 0$ that maximizes the *margin* — the perpendicular distance between the hyperplane and the nearest data points (support vectors) of each class [3]. For the multiclass case, it uses a one-vs-one decomposition internally [5].

The optimization problem with soft-margin (allowing some misclassification) is:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^N \xi_i \quad (3)$$

$$\text{subject to: } y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \quad (4)$$

Here, C is the *cost* parameter:

- **Small C** : Wider margin is preferred; more misclassifications are tolerated. The model is more robust to outliers (lower variance, higher bias).
- **Large C** : The model tries to classify all training points correctly; the margin narrows (lower bias, higher variance, risk of overfitting).

This geometric interpretation connects directly to the SVM scenarios presented in [3]: SVM first prioritizes correct classification, then maximizes the margin, and is inherently robust to outliers because only support vectors define the decision boundary [5].

2) *Hyperparameter Search Space*:

- $C \in \{10^{-4}, 10^{-3.5}, \dots, 10^4\}$ (17 values, log-spaced)
- $\text{tol} \in \{10^{-4}, 10^{-3}, 10^{-2}\}$
- $\text{loss} \in \{\text{'hinge'}, \text{'squared_hinge'}\}$
- $\text{class_weight} \in \{\text{None}, \text{'balanced'}\}$

This yields $17 \times 3 \times 2 \times 2 = 204$ combinations. The `max_iter` was set to 10,000 to ensure convergence.

3) *Effect of Cost Parameter C on Linear SVM*: As shown in Fig. 1, the SVM exhibits a different sensitivity profile compared to MLR. At very low C values (10^{-4} to 10^{-3}), CV accuracy is around 93–94%, indicating that even a wide-margin SVM achieves reasonable separation on this linearly separable dataset. The model improves steadily and reaches its optimal at $C \approx 3.162$, achieving a mean CV accuracy of 98.62% ($\sigma = 2.76\%$) with hinge loss. At larger C values (> 10), accuracy stabilizes but does not improve further, suggesting the data is nearly perfectly linearly separable once features are properly scaled.

IV. COMPARATIVE RESULTS

A. Hyperparameter Optimization Results

Table I summarizes the best hyperparameter configuration found for each model via 10-fold cross-validation on the 80% training set.

TABLE I
HYPERPARAMETER OPTIMIZATION RESULTS (10-FOLD CV ON TRAINING SET)

Model	Best C	Best Loss	Mean CV Acc.	Std CV Acc.
MLR	0.0100	N/A	98.62%	$\pm 2.76\%$
Linear SVM	3.1623	hinge	98.62%	$\pm 2.76\%$

Both models converge to the same mean CV accuracy of 98.62%, though at starkly different optimal C values. MLR achieves this with strong regularization ($C = 0.01$), preferring a simpler, smoother decision surface. Linear SVM requires a much larger $C \approx 3.162$, indicating it needs less regularization to achieve the same cross-validation performance, and that the hinge loss outperforms squared_hinge for this dataset.

B. C vs. Mean CV Accuracy

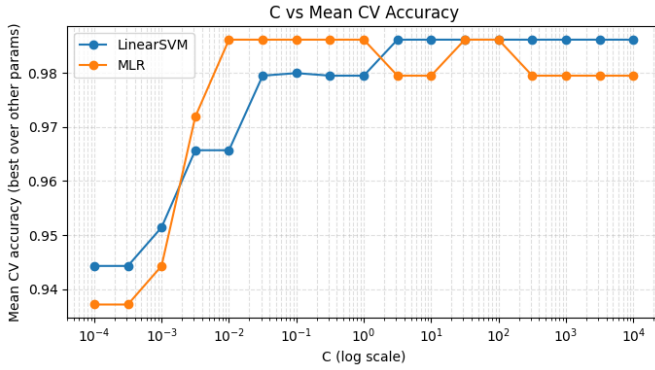


Fig. 1. Mean CV accuracy vs. regularization parameter C for MLR and Linear SVM. Each point represents the best accuracy over all other hyperparameters at that C value. MLR peaks early at $C = 0.01$ while SVM requires $C \approx 3.16$ to reach its optimum.

C. Final Test Set Metrics

Table II reports the performance of each best estimator on the 20% held-out test set (36 samples). All metrics are computed with weighted averaging across classes.

TABLE II
FINAL TEST SET PERFORMANCE METRICS (20% HELD-OUT SET, $n = 36$)

Model	Accuracy	Kappa (κ)	Precision	Recall	F1-Score
MLR	1.0000	1.0000	1.0000	1.0000	1.0000
Linear SVM	0.9722	0.9577	0.9741	0.9722	0.9720

MLR achieved **perfect classification** on the test set: all 36 samples were correctly assigned (confirmed by a diagonal confusion matrix). Linear SVM misclassified one sample — a Class 2 instance was predicted as Class 1 — resulting in a 97.22% accuracy. The Cohen’s Kappa of $\kappa = 0.9577$ for SVM still indicates almost-perfect agreement, but the gap with MLR’s $\kappa = 1.000$ is noteworthy.

D. Confusion Matrices

The confusion matrices for both models on the test set are shown in Table III and Table IV. Class labels correspond to the three wine cultivars: Class 0, Class 1, and Class 2.

TABLE III
CONFUSION MATRIX — MLR (TEST SET)

Actual \ Predicted	Class 0	Class 1	Class 2
Class 0	12	0	0
Class 1	0	14	0
Class 2	0	0	10

TABLE IV
CONFUSION MATRIX — LINEAR SVM (TEST SET)

Actual \ Predicted	Class 0	Class 1	Class 2
Class 0	12	0	0
Class 1	0	14	0
Class 2	0	1	9

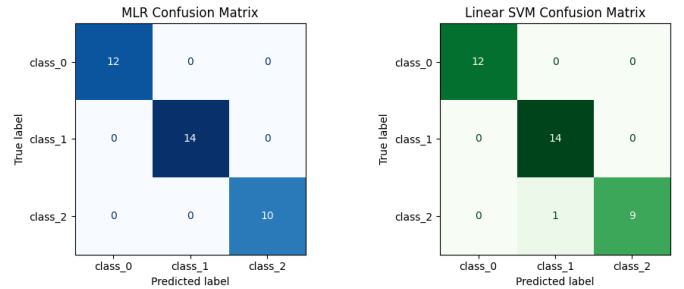


Fig. 2. Side-by-side confusion matrices for MLR (left) and Linear SVM (right) on the 20% held-out test set. MLR classifies all 36 samples correctly. SVM misclassifies one Class 2 sample as Class 1.

E. Feature Weight Comparison

Fig. 3 presents the mean absolute feature weights (averaged across classes) for both models after fitting on the full training set with their respective best hyperparameters. Larger values indicate greater influence on the classification decision.

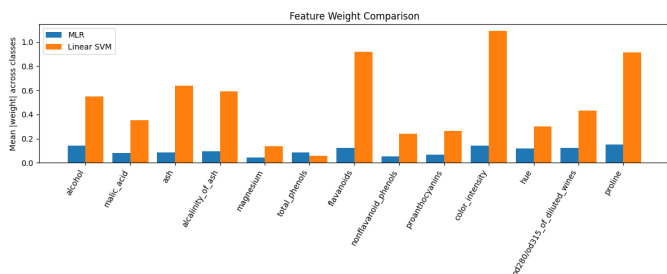


Fig. 3. Mean absolute feature weights across the three wine cultivar classes for MLR and Linear SVM. Both models assign highest importance to flavanoids, proline, od280, and alcohol, consistent with the chemometric literature on wine varietal discrimination.

Both models agree on the most discriminative features: flavanoids, proline, od280/od315 of diluted wines, and alcohol consistently receive the highest weights. This convergence across two different learning paradigms strengthens confidence that these chemical attributes are genuine discriminators of wine cultivar, not artifacts of any single model’s assumptions.

V. CONCLUSION

This study compared Multinomial Logistic Regression and Linear SVM for the task of wine cultivar classification using 13 chemical features. Both models were rigorously fine-tuned using `GridSearchCV` with 10-fold stratified cross-validation over 17 log-spaced C values and additional hyperparameters, producing experiment logs of 102 (MLR) and 204 (Linear SVM) configurations.

During cross-validation on the training set, both models tied at 98.62% mean accuracy. However, on the held-out 20% test set, MLR achieved **perfect classification** (Accuracy = F1 = κ = 1.000), while Linear SVM committed one misclassification (Accuracy = 97.22%, κ = 0.958).

The key findings are:

- 1) **MLR is more robust for this chemical dataset.** The probabilistic softmax approach generalizes slightly better than the geometric margin maximization of SVM on the final test set, despite equivalent cross-validation performance.
- 2) **MLR favors stronger regularization.** Its optimal $C = 0.01$ suggests a simpler, heavily regularized decision surface is sufficient to capture the class structure in this chemometric space.
- 3) **SVM requires larger C and hinge loss.** The optimal $C \approx 3.162$ with hinge loss indicates SVM needs a narrower margin to correctly classify this data, consistent with the tight class clustering after scaling.
- 4) **Feature agreement validates chemical discriminators.** Both models independently identify flavanoids, proline, od280, and alcohol as the most informative features, reinforcing their established role in wine chemometrics.
- 5) **Overfitting is observable in MLR at high C .** Training accuracy reaches 100% at $C \geq 1$ while CV accuracy

plateaus — a practical demonstration of the overfitting phenomenon discussed in the course lecture.

In conclusion, for linearly separable, well-scaled chemometric data with a moderate number of features and samples, MLR with appropriate L2 regularization is the recommended approach [4]. Its probabilistic output provides not only predictions but also confidence scores, offering richer interpretability for domain experts. Future work should explore nonlinear kernels and ensemble methods [6] to further characterize the decision boundary in this feature space.

REFERENCES

- [1] Scikit-learn developers. (n.d.). *API Reference: LogisticRegression, LinearSVC, Pipeline, GridSearchCV, StandardScaler, and load_wine*. Scikit-learn. Retrieved from <https://scikit-learn.org/stable/api/index.html>
- [2] UCI Machine Learning Repository. (n.d.). *Wine Dataset*. UC Irvine Machine Learning Repository. Retrieved from <https://archive.ics.uci.edu/dataset/109/wine>
- [3] Raga, R. C. Jr. (2026). *Lecture 9 – Intro to MLR and SVM* [Class lecture notes/slides].
- [4] Geetha, T. V., & Sendhilkumar, S. (2023). *Machine Learning*. Chapman and Hall/CRC. <https://doi.org/10.1201/9781003290100>
- [5] Zhang, T. (2023). *Mathematical Analysis of Machine Learning Algorithms*. Cambridge University Press. <https://doi.org/10.1017/9781009093057>
- [6] Supriatna, D. J. I., Saputra, H., & Hasan, K. (2023). Enhancing the red wine quality classification using ensemble voting classifiers. *Infolitika Journal of Data Science*, 1, 42–47. <https://doi.org/10.60084/fjds.v1i2.95>